When Do Language Servers Help Coding Agents?

Ian Barber

July 2026

Abstract

Coding agents are increasingly given language-server tooling, such as go-to-definition, compact definition spans and type diagnostics, on the assumption that better code intelligence makes a better agent. We tested the assumption against a capable text baseline of grep, ranged reads and a shell, using 7 models over constructed dispatch and seeded-defect suites, retrieval tasks built on real library source, and a shell agent on real repositories.

Making the tool available rarely helped by itself. Go-to-definition changed nothing about what the agent found when the receiver's type was written in code the agent already had open, because it reads the type there and locates the target itself. Lookup improved resolution only where the type lived outside that code, in a generated stub or behind a vendored or compiled boundary, and even then only when the agent was told to use it.

Compact definition spans cost less than reading the file, 3.35x to 4.12x less than a whole-file read and 1.30x less than grep with ranged reads, but only when the span replaces the read. Usually it did not. Handed a definition, models opened the file anyway on 35 of 36 instances, and neither telling them the span was complete nor letting them ask for it themselves changed that. Training did. After 39 demonstrations a 27B model stopped rereading, and what it learned was conditional: where the span lacked the defect it still went looking every time, and once where the untrained model went three times.

Running a type checker at the end of the agent's turn, as a gate that rejects a defective submission and asks for a repair, took accepted correct outcomes from 1 of 12 to 11 of 12 on seeded defects. The same diagnostic delivered during editing changed nothing, on a channel the model exercised in 1 of 12 tasks. Across all three services the benefit tracked whether the agent's behaviour changed, and making the tool available did not by itself change it.

1 Introduction

A language server answers questions about code. A coding agent with grep, ranged reads and a shell already answers most of the same questions on its own. What a server adds therefore depends on three things: whether it answers a question the agent cannot, whether its answer is cheaper, and whether it arrives at a better moment. Prior work rarely measures against an agent that already reads code well, so that is the baseline we use throughout.

IF: does delivering the semantic information change what the agent finds? Typed Holes [2] and LSPRAG [3] push types and definitions into the context on the premise that the information is missing, and that premise is what this question tests. A yes is the semantic arm reaching targets the text arm misses. A no is both arms finding the same targets.

FORM: does a compact span cost fewer tokens than reading the file? CodeStruct [4] makes the neighbouring case that structured reads and edits beat whole-file handling by exposing program structure. A yes is fewer total tokens at unchanged success. A no is the agent paying for the span and then reading the file the span was meant to replace.

WHEN: does the moment a diagnostic arrives change the outcome? STALL+ [5] finds static-analysis value varying with integration phase, and CoCoGen [1] checks a coherent draft before retrieving context to repair

it. A yes is the same diagnostic producing a different accepted result depending only on when it fires. A no is the same outcome whenever it arrives.

2 Method

We evaluated in two settings. The first is a purpose-built harness in which a model drives a fixed loop of six actions over a Python workspace: grep, ranged read, whole-file read, line edit, run tests, and submit. Each arm adds or withholds a single semantic operation and holds the rest constant, isolating each operation's contribution. The second is an off-the-shelf shell agent, mini-swe-agent driving Claude Sonnet 4.5 with ordinary shell primitives, run on 3 SWE-bench tasks in sympy, astropy and sphinx with an AST-backed definition command available, one seed and a 60-step cap. That setting sacrifices control for realism and we treat it as a case study throughout. Nothing was tested through an MCP server, a skill or an editor plugin, so these experiments can say which operation helps but cannot rank delivery surfaces against each other.

We created five task suites. The dispatch suite has 15 synthetic tasks in which a typed receiver calls one of roughly 10 identically named overrides, only one of which is buggy. It runs across a ladder that moves the receiver's type progressively further from the call site, from a call-site annotation to the test's construction site to factory indirection, and a fourth rung that hides the type altogether by withholding the application and test source and redacting the receiver construction. Two suites measure retrieval cost: one is fully synthetic, and the other builds constructed misuse tasks over vendored library source from toolz 1.1.0 and more-itertools 11.1.0. Another supplies mechanically validated definition-span instances for the substitution experiments. The checker suite has 12 pairs of seeded defects and clean controls, where each defect is coherent (the target file parses and leaves no unimplemented stub), passes the visible test, fails a held-out test, and carries a single semantic diagnostic in the target file, and each clean control is that defect's own validated gold. We built the tasks because a bounded scan of real repositories produced no candidate that passed all four screens: environment, leakage, ambiguity and discriminating lookup.

The definition operation in the retrieval experiments is a static AST resolver rather than a live language server. Live Pyrefly served the dispatch lookup arms and one 7B run.

We report five measures. Resolution is whether the agent fixed the intended target and passed a held-out test written against the specification, separate from the visible one. Cost is total tokens, input plus output, over the whole trajectory. Election counts how often the agent chose a semantic operation when one was offered. Substitution counts reads of the defining file after a definition span had already been delivered, which is the behaviour that decides whether a span saves anything. For the checker experiments we also report wall time, since a diagnostic that prevents a bad submission trades latency for a repair. The unit of analysis is the task, and intervals are task-level bootstraps.

The dispatch ladder, the definition-span instances, the paired retrieval comparison and the checker grid ran at temperature 0 with a single rollout per cell. The hidden rung and its matched visible control ran at temperature 0.7 over 5 seeds per task, 450 rollouts in all. Temperature 0.7 with repeated seeds also covered the whole-file contrast, the election and execution-feedback runs, and the training harvest, whose held-out retest ran at temperature 0. We ran Qwen2.5-Coder-7B, Qwen3.5-27B and Qwen3.6-27B locally, and Claude Sonnet 4.5, DeepSeek v3.1, GLM-4.6 and GPT-5.6 "Luna" through an API.

Two limitations follow from this design. The workspaces are small enough that an agent can read any file it wants under budget, so the results apply only to the regime where reading is cheap. Static tooling also reaches only what a type checker sees, which leaves out dynamic behaviour, wrong annotations and `Any`-heavy boundaries. All code, task definitions and run artifacts are at https://github.com/ianbarber/lsps-for-llms.

3 Results

3.1 Does delivering the semantic information change what the agent finds?

Where the receiver's type was visible in the source, semantic lookup made no difference. Text search resolved all 15 dispatch tasks, and go-to-definition resolved 14 unprompted and 15 when the agent was told to use it. Moving the type further from the call site did not change that: resolution stayed at 14 or 15 across the 3 ladder rungs, including the factory-indirection rung where only a type-aware server can resolve the receiver statically. The trajectories explain the flatness. The agent grepped about 0.3 times per task, read the receiver's type wherever it happened to appear, and opened the buggy override first.

Cost was flat for a more specific reason, and it is the same one that governs the retrieval results below. On matched successes, lookup cost 1.03 times text search when the tool was merely available and 0.96 times when the agent was prompted to use it, a range of 0.94 to 1.06 across all three rungs. The prompted agent

replaced its file read with the span, taking 0.07 whole-file reads per task against text search's 1.00, and came out slightly ahead. The unprompted agent called the tool and then read the file as well, 1.00 whole-file reads on top of 0.80 lookups, and came out slightly behind. Delivering a definition is only free if it displaces something.

The answer changes when the type is not there to read. The fourth rung models that case, keeping the type out of the source the agent can open so that reaching it requires a lookup, and we ran that rung at 5 seeds per task against a matched visible control. The arms are text search alone (grep_base), lookup available but unprompted (defn_avail), and lookup available with an instruction to use it (defn_prompt).

Arm	Resolved	Difference vs grep	Greps	Whole-file reads
grep_base	70/75	—	1.52	2.17
defn_avail	71/75	+0.013 [−0.040, +0.067]	1.47	2.04
defn_prompt	75/75	+0.067 [+0.013, +0.133]	0.36	0.68

Per-task resolution rates over 5 seeds at temperature 0.7, with 95% task bootstrap intervals over the 15 tasks. Greps and whole-file reads are per-task means. In the matched visible control every arm resolved all 75 rollouts.

Prompting the agent to use lookup improved 4 tasks and worsened none. Offering the same operation without prompting improved 3 and worsened 2, which leaves it indistinguishable from text search. Every arm solved every rollout in the visible control, so the paired difference there is zero and the effect exists only where the type was withheld. Tokens barely moved: among tasks both arms solved, lookup cost 0.983 times text search when prompted and 1.005 when merely available. The cost of hiding the type showed up in resolution. This rung is constructed, and it withholds source a real agent could usually open.

A separate pilot tested the resolver in isolation from the agent, on repositories that are byte-identical apart from one stub. Typed lookup reached the intended override among 8 to 15 same-named candidates. The erased variant resolved instead to the base class's declaration of the method, which every override shares, so it identified none of them, and both variants type-checked cleanly. That precision bought nothing at the agent level. All 12 task-condition cells passed, the typed automatic arm cost 1.037 times the textual arm with an interval [0.988, 1.093] that is too wide to call equivalence, every automatic result was still followed by a read of the target file, and adding the lookup step cost about 6 seconds per task.

3.2 Does a compact span cost less than reading the file?

Against a whole-file read the span is much cheaper. It cut total tokens by 3.35x for Qwen3.6-27B, 3.49x for Sonnet 4.5 and 4.12x for DeepSeek v3.1, and no attempt failed under either interface for any model.

A capable agent rarely reads a whole file, so the harder test gives the text arm grep, ranged reads and a whole-file fallback. On 11 tasks over real library source, Qwen3.5-27B spent 1,602 mean total tokens with text retrieval and 1,235 with definitions, a paired ratio of 1.297 with a bootstrap interval [1.093, 1.527], and definitions were cheaper on 10 of the 11. Both arms solved everything. It is also the case where the span did replace the read, with none of the 11 results followed by a reread, and that turned out to be the exception.

The saving therefore depends on two agent behaviours. The first is election, choosing the operation at all, and it is the tractable one. An untrained 7B invoked the definition operation in 1 of 48 rollouts; after relabel training it invoked it in all 48, stopped reading files entirely, and moved from 3,086 to 687 mean input tokens while resolution rose from 31 to 48 of 48 rollouts. Prompt framing raised use of the equivalent command on 2 of 3 tasks in the shell-agent case study. A second pair of runs isolated policy from backend: with the tool enabled in both arms, the untrained 7B never invoked it across 24 rollouts and the trained model invoked it every time, moving mean input from 2,894 to 689 tokens and success from 14 to 24. A live Pyrefly backend reproduced the saving once the policy elected, but no arm held the policy fixed while varying the backend, so what this measures is the policy.

The second behaviour is substitution, using the span in place of the file, and it failed almost everywhere. When the span was pushed into the context unsolicited, models opened the defining file anyway on 35 of 36 instances. Telling them explicitly that the span was complete and the file should not be opened removed 2 of those and cost the 27B 294 extra tokens per task.

Model	Reread, pushed span	Reread, plus sufficiency instruction
Qwen3.6-27B	11/12	12/12
Sonnet 4.5	12/12	12/12
DeepSeek v3.1	12/12	10/12

Letting the model ask for the span does not fix this. A separate run with Qwen3.6-27B, GLM-4.6 and GPT-5.6 "Luna" reproduced the pushed reread at 35 of 36, and among the instances each model chose to request the span, it still reread on 9 of 9, 4 of 9 and 7 of 12. GLM-4.6 solved 6 of these 12 tasks in the pushed arm and 5 of 12 in the elected arm, and the reread was counted independently of whether the task succeeded.

The behaviour is not an artifact of the constructed apparatus. In the shell-agent case study, a manual cross-check of the trajectories matched about 16 of 18 definition calls to a later read of the file just resolved, in one case 22 ranged reads of the same sympy module. The same agent took between 0 and 3 whole-file reads in 44 to 60 actions, which is why the large whole-file ratios above do not describe its situation.

Training was the only lever that moved substitution, and it moved it completely. A DAgger-style relabel run [6] harvested 39 demonstrations, none of which required a read, and cut the reread from 11 of 12 to none on held-out instances built from different seeds and different templates, moving mean input from 1,157 to 748 tokens for a saving of 1.59x on tasks both arms solved. Held-out pass moved from 11 to 10 of 12: 1 rescue against 2 losses that pass the visible test and fail the specification. At 12 instances and one seed, that difference is not distinguishable from noise. We then ran a pre-registered confirmation on a reserved set of 12 instances, disjoint from the harvest in both seeds and templates. It reproduced the effect: the reread fell from 12 of 12 to 1 of 12, with 11 removed, 1 persisting and none induced, mean input fell from 1,223 to 729 tokens for a saving of 1.72x on tasks both arms solved, and held-out pass moved from 12 to 11 of 12.

Because every training example had a span that already contained the defect, this leaves open whether the model learned to judge sufficiency or simply to stop reading. To separate those, we built 12 instances that hoist each override's arithmetic into a module-level helper. The span the server returns for the call site is still the complete definition of the method that binds. The defect now sits outside that span, so what the agent is handed no longer carries the line it has to change. A twin repository differing in exactly one line restored sufficiency, and a third arm left the repository byte-identical while delivering a second genuine lookup at the helper.

Reads after span	Span insufficient	Span sufficient	Helper also delivered
Untrained	12/12	12/12	11/12
Trained	12/12	2/12	0/12

The trained model went looking every time the span lacked the defect, and largely stopped when it did not. The untrained model read in all three situations, so it never made the distinction at all. Both conditions repaired every instance in every arm, on the visible and the held-out test, which means the manipulation moved retrieval behaviour without moving outcomes. Where reading was necessary the trained model read once on average against the untrained model's 3.42 times, and where it was unnecessary, 0.17 and 0.00 times against 1.58 and 1.75. Input tokens were lower in every arm. Two apparatus checks matter for reading this result. A scripted adversary searched 17 one-parameter repair forms and found none that reached the held-out test without retrieval, so the instances cannot be solved by guessing. The harness re-shows any file the agent has edited, which would have delivered the hidden line without a read, so we redacted that view while leaving deliberate reads and grep available.

3.3 Does the moment the diagnostic arrives change the outcome?

We compared 3 delivery points against a no-checker control: a diagnostic after every edit, a single diagnostic at revision, and a gate at submission that rejects a defective draft and asks for a repair. Accepted submissions that were both type-clean and correct against the held-out test rose from 1 of 12 with no checker to 10 of 12 at revision and 11 of 12 behind the gate, effects of +0.375 [+0.250, +0.500] and +0.417 [+0.292, +0.500]. Delivery after every edit stayed at 1 of 12, an effect of +0.000 [−0.125, +0.125], but that arm hardly ran: the model edited before submitting on only 1 of 12 defects, so the checker fired once. It is unexercised rather than refuted.

12 seeded defect/clean pairs	Control	After every edit	One-shot	Gate
Bad completion accepted	11/12	11/12	2/12	1/12
Revision tokens, defect	787	754	1,210	1,380
Wall time, defect (s)	27.8	14.5	68.3	62.7
Revision tokens, clean	591	—	619	591
Wall time, clean (s)	9.4	9.9	9.6	9.5

After-every-edit fired in 1 of 12 defect rows; mean edits 0.08 against 0.17 in control.

Cost tracks how often the checker speaks. On the clean drafts, which needed no diagnostic at all, the after-every-edit arm cost 651 tokens and the revision arm 619, against 591 for both the control and the gate. No arm produced a false rejection and all 12 clean drafts were accepted everywhere, so the extra tokens are the price of unconditional delivery: the two earlier arms hand over a diagnostic whether or not the draft needs one, while the gate fires only on a defective submission. On clean work the gate therefore matched control in both tokens and wall time, and it spent about 2.2 times control's wall time on defective work. It rejected 10 of 12 submissions, and every rejection ran the full repair, retest and resubmit cycle; the remaining 2 defects were repaired before the model submitted. Its one-task advantage over revision delivery came from refusal: on one task the model received the diagnostic, made no edits, and submitted anyway, and only the gate's rejection forced a repair. Its single miss is instructive in the other direction, since the model self-repaired into a state that was type-clean and still failed the held-out test, which is something no type-checker gate can catch.

Two caveats bound this. The zero false rejections on clean drafts are an apparatus check: each clean control is that defect's own validated gold and the checker is deterministic, so it could not have rejected them, and the rate on real work is unmeasured. The defects also had to be seeded because capable models rarely leave checker-detectable ones: frontier models passed every attempt, no natural 7B draft was coherent enough to use, and the coherent 14B drafts were already type-clean. A separate grid on execution feedback, covering 2 frontier models, 14 tasks, 3 seeds and 3 delivery modes, passed all 252 of its attempts.

4 Discussion

4.1 Delivery

Where the receiver's type was visible in the source, semantic navigation added little over grep and ranged reads, because the lookup returned what the model could already derive: it read the receiver's type in the code it had open, worked out which override the call bound to, and went to the right file on its own. The type being present and correct in the source is what carries resolution; the server only queries it.

Navigation helps when the type lives outside the source the agent can open, in a stub, a compiled extension or a vendored package. The agent then has to fetch the type, and lookup resolved tasks that text search could not. Even there, availability was not enough: the agent had to be told to use the operation before resolution moved. Sound types also sharpen the resolver itself, picking the right override out of many that share a name, but that precision produced no gain at the agent level.

4.2 Form

A compact span is cheaper than a read only when it replaces the read, and whether it does is decided by the agent's policy. Against a whole-file read the span wins by a wide margin, but a real coding agent rarely reads a whole file in the first place, so the margin it works with is the narrow one against competent grep and ranged reads.

Two behaviours sit between the operation and the saving, and they proved very different. Election, choosing the operation at all, moves easily: prompt framing shifted it on capable models and training shifted it on a small one. Substitution, using the span instead of opening the file, resisted pushing, explicit instruction and self-election alike, in the constructed tasks and in a shell agent working on real repositories.

Training moved substitution where nothing else did, and what it produced was a judgment rather than a reflex. Given a valid span that did not contain the defect, the trained model went looking every time, and read once on average where the untrained model read 3.42 times. Given a span that sufficed it mostly stopped, and given the missing definition as well it stopped entirely. Correctness did not change in any of these conditions, so what training bought is an agent that retrieves only when retrieval is warranted, though we tested that judgment against a single form of insufficiency. It learned that distinction from demonstrations in which reading was never the right move.

4.3 Timing

With the type checkers we have today, the end of the turn is the right place to run one. Of the two late options the gate is the better deal, because it charges only defective work: diagnostics at revision arrive whether or not the draft needs them, so every clean draft pays, while clean work behind the gate costs what it did without a checker. Delivering the same checker after every edit changed nothing on a channel that fired on 1 of 12 defects, so that arm tells us nothing either way.

The gate's ceiling is the checker behind it, since a submission can clear a type check and still be wrong, which is what happened on the one defect it let through. The opportunity is scarce in any case, because capable models rarely leave a checker-detectable defect at all. All of this bounds the claim to today's checkers; whether a better-timed one could do more is open.

5 If you use a coding agent

Run your type checker at the end of the agent's turn and make it blocking. This is the clearest return in the report and the cheapest to wire, because most harnesses expose a hook at that point. It costs nothing when the agent's work is already clean, since a passing check produces no output and no repair. To see whether it is earning anything on your codebase, count how often it blocks and how often the resulting repair passes.

Do not add a navigation tool if your types are already written where the agent reads. The test is one you can run by eye: open a file your agent works in and ask whether you could tell what type a variable holds from what is on the screen. If you can, so can the model, and it will find the right definition without help.

Consider one where you cannot. The cases that matter are the ones where you would have to jump elsewhere to answer that question: types that live only in a generated stub, behind a compiled extension, inside a vendored package, or on an object assembled somewhere else. Installing the tool is not sufficient on its own. In our tests the arm that merely made the operation available performed like plain text search, and only the arm that told the agent to use it improved anything, so put that instruction in your project instructions or the tool description.

Do not assume a definition tool saves tokens until you have checked that the agent stops reading. Count reads of the defining file that happen after the tool has already returned it; if that number is not near zero, the tool is adding context on top of the read it was meant to replace, and your token bill will go up.

Fine-tuning works, but only if you own the weights. That rules it out for a hosted agent. Where it is available it buys a conditional policy: the trained model still went looking every time what it had been handed was insufficient.

Keep type annotations correct and present, since that is what the text baseline exploited. On your own workload, measure wrong-file edits, post-span reads of the defining file and spurious gate rejections, because the clean-work cost here came from workspaces guaranteed clean.

6 Future work

Retrieval calibration is the line most worth following. A harvest of 39 demonstrations, none of which required a read, produced a model that goes looking when what it holds is inadequate and stops when it is not. That is a judgment about the sufficiency of retrieved context, learned from examples that never exercised it. Nothing in that is specific to definition spans. Agents retrieve constantly and mostly indiscriminately: grep hits, file chunks, documentation, search results, the output of other tools. If "do I have enough?" is trainable, it is a general lever on agent cost, and the open questions generalize with it. Does the judgment transfer across retrieval channels, or was it learned about one? Does it survive forms of inadequacy structurally unlike the delegation tested here, or was the model reading a surface cue? Does it hold at other scales, outside a task-specific adapter, and on a real codebase, where sufficiency is a matter of degree?

A second direction is co-designing the tool with the agent. Every tool tested here was built for a human in an editor. A type checker reports on every change because a person can glance at a squiggle and ignore it,

and a language server answers the question it was asked because a person chose to ask. An agent inherits both defaults and can only take all of the output or take it at one moment it picks, which is why the best available option was to run the checker once at the end of the turn. A checker built for an agent could decide when a diagnostic is worth interrupting for, suppressing what the agent is already about to fix and breaking in when it is heading somewhere expensive; a server built for an agent could anticipate what the caller needs next. The deeper question is whether the tool can learn when to speak. Nothing here tests that, and the gate result is as far as the human-facing version goes.

Can a codebase be assessed in advance? Our results say the discriminator is whether the type is readable in the source the agent already opens. That is a property of the repository, and yet a practitioner has no way to compute it. Something like annotation density, the depth of indirection between a call site and the definition that binds it, or the share of types that resolve only through a stub or a compiled boundary might predict how much a language server buys before anyone installs one. That would turn a per-workload measurement into a decision that can be read off the repository.

Finally, which tasks does semantic tooling help, and which does it hurt? Everything here is dispatch ambiguity and small seeded defects in workspaces small enough to read. The intuition worth testing is that semantic tooling matters in proportion to how much relevant context sits outside what the agent can afford to read, which would point at wide refactors, cross-module renames and API migrations, well beyond the localised bug fixes studied here. The opposite end deserves attention too: an agent that stops reading is cheaper until the day the thing it needed was in the part it skipped, and we do not know what class of task that failure lands on.

7 Conclusion

Of the three services we tested, one paid off plainly. Running a type checker at the end of the agent's turn, as a gate that blocks a defective submission and asks for a repair, was cheap to wire, cost nothing on clean work, and turned most bad submissions into repaired ones. Go-to-definition mostly told the agent what it could already read, and earned its place only where the type sat outside the source at hand, and then only when the agent was prompted to reach for it. Compact spans do cost less than reading a file, but only if the agent stops reading the file, and mostly it did not. Fine-tuning removed that habit on one model in one apparatus, which establishes that the behaviour is trainable without saying much about how to obtain it in an agent whose weights you do not control. The common thread is that the tooling moved outcomes only where it changed what the agent did, and none of it changed that by being available.

References

[1] Zhangqian Bi, Yao Wan, Zheng Wang, Hongyu Zhang, Batu Guan, Fangxin Lu, Zili Zhang, Yulei Sui, Hai Jin, and Xuanhua Shi. Iterative refinement of project-level code context for precise code generation with compiler feedback. In Findings of the Association for Computational Linguistics, 2024. CoCoGen; arXiv:2403.16792.

[2] Andrew Blinn, Xiang Li, June Hyung Kim, and Cyrus Omar. Statically contextualizing large language models with typed holes. Proceedings of the ACM on Programming Languages, 8(OOPSLA2), 2024. arXiv:2409.00921.

[3] Gwihwan Go, Quan Zhang, Chijin Zhou, Zhao Wei, and Yu Jiang. LSPRAG: LSP-guided RAG for language-agnostic real-time unit test generation. arXiv preprint arXiv:2510.22210, 2025.

[4] Myeongsoo Kim, Joe Hsu, Dingmin Wang, Shweta Garg, Varun Kumar, and Murali Krishna Ramanathan. CODESTRUCT: Code agents over structured action spaces. arXiv preprint arXiv:2604.05407, 2026.

[5] Junwei Liu, Yixuan Chen, Mingwei Liu, Xin Peng, and Yiling Lou. STALL+: Boosting LLM-based repository-level code completion with static analysis. arXiv preprint arXiv:2406.10018, 2024.

[6] Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS), 2011. PMLR v15:627–635.